ENGINEERING BLUEPRINT / SECOND EDITION

AI-Driven Product Development Lifecycle

Proof-first AI review + Release Confidence + Architecture Drift Radar

01 INTENT — 02 DESIGN — 03 BUILD — 04 PROVE — 05 REVIEW — 06 RELEASE — 07 LEARN

MANDATORY INDEPENDENT EVIDENCE GATE BEFORE MERGE

PROOF-FIRST REVIEW

RED → GREEN

Defect claim → failing test → minimal fix

RELEASE CONFIDENCE

IMPACT → PROVE

Risk-linked suites, failure modes, evidence

ARCHITECTURE RADAR

BASELINE → DIFF

Ownership, dependency and behavior drift

North-star outcome

Every pull request carries trusted, reproducible evidence bound to its commit; every material financial, architectural or operational decision remains under accountable human control.

Built for GitHub, Devin, Copilot and Java payment platforms. This second edition integrates the new **red/green evidence-gated review loop**, a risk-aware **Release Confidence engine** and a continuously maintained **architecture drift graph**.

Document status: implementation proposal, not a statement that these capabilities already exist. Configuration fragments are templates and require security and architecture validation.

RECOMMENDATION

1. Strategy and integrated operating model

Implement ADPLC as a governed engineering platform, not a collection of AI prompts. Code generation can scale rapidly; risk ownership, architecture compliance and reproducible proof must scale with it.

Non-negotiable rule

AI may propose a defect, a fix, a test or an architecture change. It cannot self-certify correctness, fabricate a passing report, edit its own gate policy or approve an exception.

The five platform engines

Engine	Responsibility	First implementation
Specification	Jira intent, acceptance criteria, positive + prohibited outcomes	Versioned YAML/JSON contracts
Proof-first Review	Red test, narrowly scoped fix, green rerun and adversarial validation	Test runner + GitHub PR template
Release Confidence	Select required checks from business and technical impact	Cucumber/JUnit suite catalogue + rules
Architecture Guardian	Check owned responsibilities, dependencies and contracts	ArchUnit + graph diff + schema verification
Evidence & Decisions	Verify origin/freshness of proof and retain justified exceptions	Trusted GitHub Actions + immutable artifact references

The lifecycle

Step	Output	Merge or release condition
01 Intent	Business outcomes, forbidden outcomes, risk assumptions	Acceptance criteria and ownership are explicit
02 Design	Impacted services, contract effects, design decision record	Architecture owner approves material shifts
03 Build	Bounded Devin/Copilot changes	Scope remains consistent with approved design
04 Prove	Red-to-green proof, RC suite, architecture diff	All required evidence generated by trusted CI
05 Review	Independent verification, concise reviewer pack	Correct human owner accepts residual risk
06 Release	Canary/controlled rollout and rollback policy	Pre-release and operational criteria pass
07 Learn	Incidents, false positives, escaped defects	Rules/tests/decisions improve over time

PILLAR 1 - DEFECT PROOF

2. Evidence-gated AI code review

The loop in the supplied diagram is preserved and strengthened: an agent identifies a candidate defect, a failing test demonstrates the failure, and only a bounded fix that makes the test pass is permitted.



NO REPRODUCIBLE FAILURE? NO DEFECT FIX.

For features: use failing acceptance criteria; for design risks: require policy evidence and approval.

Required sequence for defect-fix PRs

Gate	Implementation rule	Evidence
Identify	State the faulty behaviour and expected invariant before editing	Issue + affected scenario + impact
RED	Write a meaningful regression test; run against the unmodified baseline	Recorded failing test and baseline SHA
GREEN	Apply the smallest justified change; rerun regression + affected suites	Passing test report and candidate SHA
CHALLENGE	Run independent negative controls and relevant historical regressions	Test run links + reviewer-visible diff
VERIFY	Independent CI confirms reports are real, current and policy-complete	Validated evidence manifest

Scope limitation - important

“No failing test? No code change” is a strong defect-fix discipline, **not** a universal rule. New functionality starts with unmet acceptance tests; architecture and non-functional changes require dependency, contract, performance or security evidence, often plus human design approval.

Anti-gaming: reject tests that assert the implementation rather than the business outcome; prevent the agent from disabling assertions, deleting regression coverage or changing gate configuration in the same unapproved PR.

PILLAR 2 - IMPACT-BASED ASSURANCE

3. Release Confidence engine

Release Confidence (RC) is the testing and decision engine for **what must be proven** before a payment change is merged and released. It should reuse existing Cucumber, JUnit, contract and recovery tests rather than create a second test framework.

Core algorithm

Changed files + changed contracts + Jira intent + architecture graph → impacted capabilities / flows → risk tier → mandatory scenarios → trusted test execution → coverage + residual risk report.

Profile	Use when	Required evidence
RC-0 / Base	Documentation or truly non-behavioural edits	Build, policy integrity, basic quality
RC-1 / Local	Small service-local changes	Unit, static analysis, boundary checks, focused functional
RC-2 / Integrated	Payload mapping, integrations, public contracts	RC-1 + impacted Cucumber + contract tests + representative E2E
RC-3 / Critical	FSM, posting, clearing, idempotency, recovery	RC-2 + fault injection, replay/duplicate, ordering and restart scenarios
RC-4 / Release	Deployment or multi-service release candidate	Cross-flow regression, rollback rehearsal, observability & rollout proof

Suite-selection policy

- **Coverage map:** each requirement/invariant has stable test IDs, owner, risk class and product-flow tags (country, direction, capability, rail).
- **Impact engine:** include transitive dependencies, shared configs, message schemas, CQ/Kafka contracts and FSM transition effects.
- **Unknown means broader:** if the blast radius cannot be established, choose RC-3 or require triage; do not classify unknown as low risk.
- **Independent selection:** agent-suggested suites are advisory; trusted policy determines the required minimum.
- **Incremental execution:** run a lean PR suite, then scheduled deeper mutation and cross-flow suites with defined promotion rules.

Key output

The PR shows **required scenarios, executed scenarios, exclusions, pass/fail, freshness, known coverage gaps and risk acceptance** - not a single opaque “confidence score”.

EXAMPLE - CORRECTNESS AND RECOVERY

4. RC scenario catalogue for payments

Below is an illustrative catalogue for a payment orchestrator. It captures positive, negative and recovery behaviour. Adapt service names, identifiers and country-specific workflows to the actual platform.

Invariant / scenario	Expected proof	Minimum profile
Duplicate message ID	Same business event does not cause a second financial posting	RC-3
Timeout after external send	Retry yields one effective side effect; eventual state is consistent	RC-3
Restart in pending state	State and message journal recover without lost or duplicated actions	RC-3
Out-of-order responses	FSM accepts only legal transitions; stale callback is handled deterministically	RC-3
Sanctions/clearing rejection	Country-specific expected balance movement and terminal state	RC-3
Posting schema change	Producer/consumer compatibility + downstream reconciled outcome	RC-2/3
Validation mapping change	Country/flow scenario mapping and negative validation cases	RC-2
Rules engine update	Expected actions and prohibited side effects across selected flows	RC-2/3
Deployment rollback	Previous compatible service recovers with no orphaned transaction	RC-4

Proof to generate per scenario

Metadata	Content
Identity	Jira + requirement ID + invariant ID + test ID
Execution	Runner, target commit, environment, test framework version, timestamps
Outcome	Assertions, structured report, trace correlation ID, failure artifacts
Coverage	Impacted flows covered; known scenarios excluded and why
Audit	Artifact digest / attestation, reviewer exception (if any)

A practical guardrail

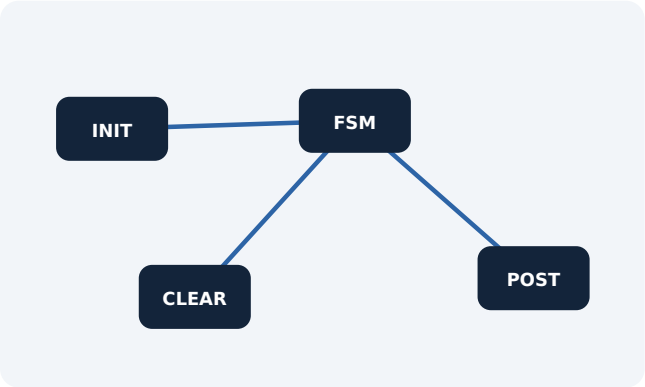
For business-critical tests, first prove the test can fail when the protected behaviour is deliberately violated. Perform mutation/fault-injection runs in a controlled isolated environment; do not disable production safeguards.

PILLAR 3 - EXECUTABLE BOUNDARIES

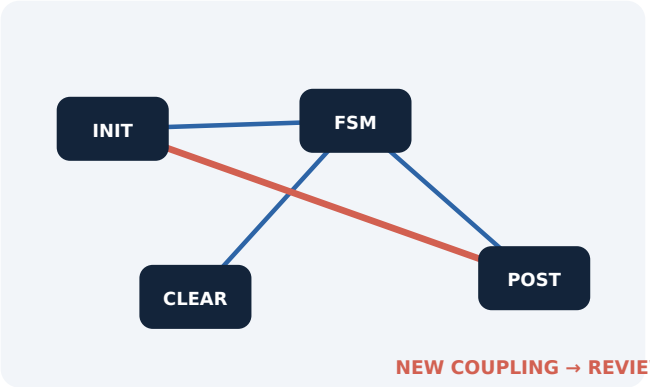
5. Architecture Drift Radar

Architecture Drift Radar continuously compares an approved service-responsibility graph with each candidate change. It catches changes that compile and pass functional tests but move rules into the wrong place or create problematic coupling.

APPROVED BASELINE



PROPOSED CHANGE



Evidence sources

Source	What the radar extracts
Java / build graph	Dependencies, package access, cycles, new coupling, ownership tags
Service integrations	REST calls, Kafka topics, CQ channels, schemas and endpoints
Domain ownership	Where validation, FSM, posting, clearing and recovery decisions live
Runtime & config	FSM YAML, reference data contracts, routing rules, deployment toggles
Design history	Approved architecture decisions, exception expiry and rejected alternatives

Two enforcement channels

Hard blockers	Soft indicators
Forbidden import or direct service dependency	Rising coupling and graph complexity
Incompatible message / API / FSM contract	Duplicate business rule in two services
Unapproved change in protected ownership	Service “churn” around a responsibility boundary
Policy weakening or unauthorized exception	Gradual increase in cross-team co-change

A soft drift indicator triggers targeted review; a documented hard-policy breach blocks merge. Do not treat a numerical “entropy” score as an unquestionable release decision.

EXECUTABLE POLICIES + ACCOUNTABLE DECISIONS

6. Architecture protection rules

Protect both **structural boundaries** (which are testable mechanically) and **semantic ownership** (which requires review of intent and behaviour).

Area	Automated checks	Human judgment required
Dependencies	ArchUnit, import graph, cycle detection	Is the dependency conceptually justified?
Service responsibility	Changed packages, topic/service ownership map	Did business logic migrate to the wrong capability?
FSM	Schema validation, legal-transition and model-based tests	Is the new business lifecycle authorised?
Integration	Schema compatibility and contract suite	Does the change preserve semantics and timing?
Recovery	Crash/retry/idempotency invariant suites	Are failure assumptions complete?
Exceptions	Policy version and approved waiver signature	Is the risk/time-limit acceptable?

Sample policy (illustrative)

```
service: payment-orchestrator
policy_version: payment-services-v2
protected_capabilities: [fsm, idempotency, recovery]
forbidden_outbound_dependencies: [posting-internals]
requires_architect_on: [new_service_edge, ownership_change, fsm_transition_change]
invariants:
  - id: PAY-NO-DUPLICATE-POSTING
    required_suite: rc-idempotency
  - id: PAY-LEGAL-FSM-TRANSITIONS
    required_suite: rc-fsm-model
on_unknown_impact: escalate
on_missing_proof: block
```

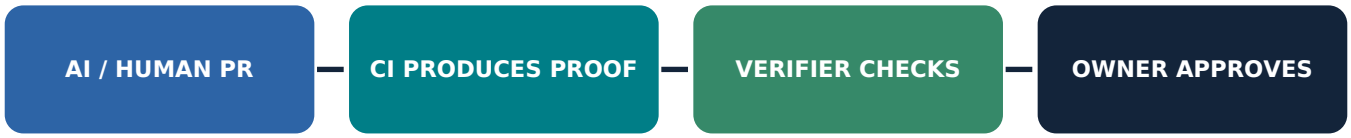
Radar build order

Start with ArchUnit + GitHub CODEOWNERS + published API/message contract diff. Add a graph of all integration edges. Finally, add semantic responsibility tagging and decision-history retrieval. The latter requires careful review to prevent false positives.

TRUST BOUNDARY AND PROOF PROVENANCE

7. Independent Evidence Gate

The mandatory evidence gate is an independent verifier, not a generated checklist. The build and tests run in governed CI; a separate verifier retrieves structured results and checks correctness, completeness and origin.



FAIL CLOSED IF A REQUIRED TEST, POLICY OR ATTESTATION IS MISSING

Verification	Block when
Commit binding	Tested head SHA or merge-group SHA differs from the candidate
Suite coverage	Required scenario or invariant cannot be matched to an executed trusted test
Test integrity	Expected fail/pass sequence absent for defect fix, or assertions disabled
Architecture	Hard boundary violation without active approved exception
Artifact authenticity	Report digest, workflow identity or attestation cannot be confirmed
Freshness	Relevant code/config/contract changed after evidence generation
Residual risk	Critical open failure lacks appropriate auditable approval

Manifest example - an index, not proof

```
{
  "pr": 284,
  "head_sha": "<tested exact SHA>",
  "run_id": "<trusted CI run ID>",
  "risk": "RC-3",
  "requirements": ["PAY-412"],
  "required_suites": ["contract", "rc-idempotency", "rc-fsm-model"],
  "reports": [{"name": "junit-results", "sha256": "<digest>"}],
  "architecture_policy": "payment-services-v2",
  "verdict": "pending-independent-verification"
}
```

Hardening: least-privileged CI tokens; centrally owned policy and workflow; pinned actions; no privileged execution of untrusted PR code; no unapproved gate edits by implementation agents.

MERGE PROTECTION AND REVIEWER ROUTING

8. GitHub enforcement pattern

Use protected branches / rulesets with mandatory pull requests, required checks, code-owner reviews and tightly controlled bypass. Prefer centrally governed required workflows when available on the organisation plan.

Recommended GitHub controls

Control	Required behaviour
Org ruleset	Applies to main and release branches; disallows force push and broad bypass
Required verifier	Cannot be green on missing or stale results; PR and merge queue aware
CODEOWNERS	Architecture owner reviews protected interfaces, policies and FSM changes
Approval policy	Stale approvals dismissed; changes to gate logic need independent owners
Merge queue	Run relevant checks on merge_group candidate if enabled
PR output	Risk tier, architecture diff, RC matrix, evidence links, reviewer questions

Illustrative per-repository caller

```
name: ADPLC Gate
on:
  pull_request:
  merge_group:
permissions:
  contents: read
jobs:
  gate:
    uses: your-org/engineering-adplc/.github/workflows/verify.yml@<pin-sha>
```

Important implementation detail

A workflow definition alone does not make a mandatory check: configure the organisation/repository ruleset to **require** the intended status. Check GitHub plan support, reusable-workflow visibility, fork trust boundaries and merge-queue behavior in a sandbox repository before enabling enforcement.

Minimum fail-closed drills

- Missing evidence → merge blocked.
- A new commit after verification → stale proof no longer certifies it.
- Attempt to edit policy without owner → review blocked.
- Forbidden dependency → architecture job fails.
- Missing critical RC suite → merge blocked.
- Failing merge-group candidate → no final merge.

REVIEW DECISIONS, NOT JUST DIFFS

9. Reviewer experience and approval rights

The reviewer should receive a one-page generated “proof packet” attached to the PR, with direct links to the authoritative test reports and exceptions. This is a summary of trusted evidence, not a substitute for it.

PR proof packet	Illustrative content
Intent	PAY-412 | duplicate payment callback | expected no second posting
Change footprint	State Manager only | 4 Java classes | 3 tests | no schema change
Proof-first	Baseline failing regression recorded; candidate passing regression recorded
Release Confidence	RC-3 required 24 scenarios / executed 24 / passed 24
Drift Radar	0 forbidden edges | no new service responsibility | no FSM schema change
Evidence	Run ID, exact SHA, test report digests and coverage gaps
Residual questions	Can a delayed duplicate arrive after journal restore?
Approvals	Service owner + architecture owner (because recovery touched)

Risk-based minimum review

Risk	Typical work	Minimum oversight
Low	Docs, isolated tests, non-behavioural	Passing gate + named human reviewer
Medium	Bounded logic or mapping change	Gate + code/domain owner
High	FSM, money movement, ordering, recovery	RC-3 proof + domain and senior architecture sign-off
Critical	Control/financial safety boundary, production incident	High-risk controls + formal release/change owner approval

Review workload control

Introduce a work-in-progress cap for agent PRs (for example two open PRs per available reviewer), smaller diffs and an “evidence ready” pre-review queue. Track **active human review minutes**, not just elapsed PR lead time.

Independence: a code-writing AI can summarise its changes, but the verifier must not be under its write access. Material architectural or payment-behaviour exceptions require qualified humans.

BEYOND CONVENTIONAL AI-DLC

10. Advanced controls worth building

Innovation	Why it matters	First safe version
Negative-space development	Defines what must never happen: duplicate posting, illegal FSM transition, lost recovery	Invariant catalogue + negative Cucumber scenarios
Test-of-the-tests	A green test can be ineffective; mutation testing demonstrates sensitivity	PIT / isolated guard-disable experiment for critical suites
Counterfactual behaviour twin	Detects unexpected changes by replaying same sanitised events against baseline and candidate	One isolated service, no real external side effects
Evidence expiry graph	A shared contract/config merge can invalidate proof from another PR	Track SHAs, schema versions and affected suites; retest
Decision / rejection memory	Stops agents recreating rejected designs and losing incident knowledge	Versioned decision records linked to ownership
Production invariant feedback	Incident traces become future RC / architecture policy tests	Observability event → triage → verified test addition

Illustrative sequencing

Value-first roadmap

1. Proof-first review + mandatory evidence verifier. 2. Release Confidence suite selection for one payment flow. 3. Architecture Drift Radar hard boundaries. 4. Counterfactual replay and test mutation. 5. Cross-PR evidence expiry and production learning.

A critical design constraint

Evidence is probabilistic assurance, not mathematical proof of correctness. The more sophisticated the automation, the more important it is to document blind spots, test the gate with intentionally failing examples and maintain independently owned policies.

How the controls reinforce each other

- **Proof-first** proves the defect is real and the intended fix addressed it.
- **Release Confidence** checks broader financial and recovery behavior beyond the single defect.
- **Architecture Drift Radar** checks that implementation has not corrupted service boundaries.
- **Independent Evidence Gate** makes all three claims verifiable and merge-enforceable.

FIRST 12 WEEKS - REALISTIC PILOT

11. Implementation roadmap

Weeks	Work package	Acceptance / exit evidence
1-2	Inventory one service, GitHub policies, baseline ruleset and verifier skeleton	Known bypasses documented; test PR with missing gate blocked
3-4	Introduce proof-first PR template, policy schema and ArchUnit baseline	RED/GREEN evidence working; forbidden import blocked
5-6	RC flow / invariant catalogue and impacted suite selection	Critical missing scenario blocks merge; traceable RC matrix
7-8	Architecture Drift Radar integration graph and semantic owner routing	New edge flagged; hard violation blocks; soft drift reviewed
9-10	Mutation sampling, isolated behavior twin, evidence expiry	Known broken guard detected; stale proof revalidated
11-12	PR proof packet, metrics dashboard, learning loop, expand pilot	10-20 PR audit; quality not worse; reviewers report useful evidence

Backlog - ordered implementation work

ID	Deliverable / acceptance criterion
ADPLC-01	Protected branch ruleset and documented bypass authorities
ADPLC-02	Centrally controlled mandatory verifier; PR + merge-group aware
ADPLC-03	Defect RED/GREEN proof record with immutable baseline / candidate refs
ADPLC-04	RC scenario catalogue tagged by service, country, flow and risk
ADPLC-05	Impact selector + unknown-impact escalation path
ADPLC-06	ArchUnit rule suite and forbidden-edge tests
ADPLC-07	Drift Radar approved-baseline integration graph + PR graph diff
ADPLC-08	Evidence manifest, digest checks, run identity + freshness verifier
ADPLC-09	CODEOWNERS, risk-based routing and waiver approval workflow
ADPLC-10	Reviewer proof packet and pilot performance measures

Pilot choice

Select one Java service with a known recovery/idempotency invariant and one representative payment flow. Start with read-only policy reporting, run failing drills, then turn on merge blocking only after false-positive and exception handling are reviewed.

OUTCOMES, NOT ACTIVITY COUNTS

12. Scorecard and acceptance criteria

Evaluate the program by accepted trustworthy changes, review effort, defect escapes, architecture integrity and compute cost. Do not optimise for PR volume alone.

Metric	Pilot measure	Initial acceptance target
Mandatory gate coverage	Protected branch merges that passed verifier	100%
RC critical traceability	Required critical invariants with valid executed evidence	Target ≥95%; investigate the remainder
Architecture escapes	Verified hard-policy bypasses after merge	0
Reviewer effort	Median active reviewer minutes per accepted PR	Aim for ≥25% improvement vs baseline
False positives	Blocked PRs with incorrect risk/policy decision	Downward trend after calibration
Escaped financial defects	Post-merge severity weighted regressions	No increase / ideally decreasing
Mutation detection	Designed defects caught by protected RC suites	Critical sampled mutations detected
Agent cost efficiency	Devin ACU or equivalent per accepted change	Downward trend without quality decline
Recovery readiness	Fault-injection / replay suite pass and completeness	100% required RC-3 scenarios

Operating ownership

Role	Accountability
Engineering manager	Pilot governance, trade-offs, reviewer capacity, targets
Service owner	Expected behavior, service-local design, runtime impact
Architect	Boundary policy, ADRs, hard drift exceptions
Quality / SRE	Release Confidence scenario integrity, fault-injection and run reliability
Platform / DevSecOps	Trusted workflow ownership, evidence verification and rulesets
AI agent	Produce bounded code/test proposals and review summaries - no self-approval

First executive demo

Show a PR where an AI proposes a duplicate-posting fix. Baseline test fails; patch passes; RC recovery suite passes; Radar finds no forbidden dependency; independent verifier passes; correct humans approve. Then demonstrate that removing one test or inserting one forbidden edge blocks the merge.

START WITH DESIGN, NOT PRODUCTION EDITS

Appendix A. Implementation brief for Devin

Paste this compact instruction into a bounded planning task. Ask the agent for a design proposal and test plan before authorising changes.

```
TASK: Implement an ADPLC pilot for one Java payments service.

First inventory: GitHub branch rules, CODEOWNERS, CI tests, current RC
framework, service boundaries, FSM/config contracts and recovery flows.
DO NOT change payment behavior or merge policies until design approval.

Build:
1) Defect RED/GREEN proof contract with baseline + candidate SHA.
2) RC impact selector and required scenario coverage matrix.
3) Architecture Drift Radar: ArchUnit + integration edge diff.
4) Independent Evidence Gate with report provenance/freshness checks.
5) Risk-based PR reviewer packet and protected owner approvals.

Safety: no synthetic PASS; no self-approval; fail closed on missing
critical evidence; policy/verifier not editable by coding agent.
Reuse existing Java, Cucumber and GitHub infrastructure.

Return: design, repo/file-level plan, minimal test PR, failing drills,
metrics, rollout order, decisions requiring human sign-off.
```

Appendix B. Pilot PR checklist

- Jira intent, service owner, acceptance criteria and prohibited outcomes recorded.
- For a defect: regression fails on baseline and passes on candidate; both executions linked.
- Release Confidence selected suites are complete and from the governed catalogue.
- Architecture Radar reports baseline/candidate graph, new edges and unapproved drift.
- Trusted verifier checks evidence SHA, run identity, artifacts, policy and freshness.
- Human approval comes from correct domain, architecture and release owner for risk.
- No unreviewed modifications to policies, test harness, required workflows or waivers.

Reference points

GitHub documentation: rulesets, required status checks, code owners, merge queues, reusable workflows, artifact attestations and Actions security. Java architecture testing: archunit.org. Mutation testing: pitest.org. Policy-as-code: openpolicyagent.org. Validate plan-specific GitHub features with your administrators.

Bottom line

AI should accelerate implementation while **slowing unsafe decisions**. Every accepted change is reviewed as **business intent + minimal implementation + release confidence + architecture integrity + trusted evidence + residual risk**.